

Boolean Arithmetic over $\mathbb{F}_2$ from Group Commutators

Marc Joye 

Zama
Paris, France
`marc@zama.ai`

Abstract. This paper studies efficient realizations of arithmetic over the binary field $\mathbb{F}_2$ in nonabelian groups using only intrinsic group operations, namely multiplication and inversion. The constructions rely on commutators to implement Boolean computation within the group structure. Two complementary approaches are presented: a realization of a universal Boolean gate (NAND) and direct realizations of the field operations XOR and AND. These approaches apply to finite nonabelian simple groups and can be implemented using a small number of group operations. Explicit realizations are provided in the alternating groups A_5 and A_6 . For the smallest nonabelian simple group A_5 , these constructions achieve state-of-the-art efficiency in the number of group operations.

Keywords: Arithmetic over $\mathbb{F}_2$ · Boolean computation · Commutators · Group-based computation · Fully homomorphic encodings

1 Introduction

The expressive power of nonabelian groups as computational media has been recognized for several decades. In algebraic complexity, Barrington’s theorem [1] shows that polynomial-size branching programs of width 5 over a fixed nonabelian simple group capture exactly the class NC^1 . Earlier work of Maurer and Rhodes [9] (see also [6]) already demonstrated that Boolean functions can be realized using finite nonabelian simple groups. These results illustrate that non-commutativity provides a rich computational structure even when only a small set of algebraic operations is available.

This observation raises a natural question: to what extent can arithmetic and Boolean computation be realized directly inside group operations? At first sight this appears unlikely, since a group provides essentially a single binary operation (together with inversion), whereas arithmetic requires both addition and multiplication. Nevertheless, suitable encodings of bits as group elements may allow both operations to be recovered from the intrinsic group structure. In particular, arbitrary Boolean circuits can then be evaluated using only the operations of group multiplication and group inversion.

To describe this phenomenon, it is convenient to adopt the viewpoint of a *fully homomorphic encoding* inside a group. Informally, elements of a ring are

represented by group elements in such a way that appropriate combinations of intrinsic group operations implement the desired arithmetic operations. When the message space is $\mathbb{F}_2 = \{0, 1\}$, this corresponds to realizing the Boolean operations XOR and AND on encoded bits using only group operations.

Related work Early examples of arithmetic over $\mathbb{F}_2$ in groups include the embedding of $(\mathbb{F}_2, +, \cdot)$ into $\text{SL}(3, 2)$ by Ben-Or and Cleve [2], the realization inside S_7 studied by Rappe [12], and the construction in the alternating group A_5 given by Khamsemanan, Ostrovsky, and Skeith [5]. Subsequent work of Nuida gave realizations in the symmetric group S_5 [11], with further refinements in [10]. More recently, Guillot et al. [4] described an explicit realization in the group S_6 , building on earlier constructions. Related perspectives appear in works investigating group-theoretic approaches to fully homomorphic encryption [3, 5, 7, 11, 4].

Our techniques and contributions The present work studies how Boolean computation and arithmetic over $\mathbb{F}_2$ can be realized efficiently using only intrinsic operations of certain nonabelian groups, namely multiplication and inversion. Our constructions rely on commutators as a basic algebraic primitive. A key structural property supporting this framework is Ore’s theorem, proved in [8], which asserts that every element of a finite nonabelian simple group is a commutator. In particular, the commutator map is sufficiently expressive to reach any element of the group, suggesting that commutator-based constructions can serve as building blocks for Boolean computation inside such groups. Related results appear in [5, Lemma 6.3], where it is shown that, for any distinct elements α, β with $\beta \neq e$ in a finite nonabelian simple group G , the element α can be expressed as a product of functions of the form $(x, y) \mapsto [g x g^{-1}, h y h^{-1}]$ evaluated at (β, β) . While this result allows arbitrary products of such commutators, the constructions considered here focus on very small products, with the main case being a single commutator (i.e., $k = 1$) and, in one instance, a product of two commutators.

We present two complementary approaches: (i) a commutator-based realization of a universal Boolean gate (NAND), and (ii) direct realizations of the arithmetic operations XOR and AND over $\mathbb{F}_2$. Both approaches rely solely on multiplication and inversion in the underlying group and apply to finite nonabelian simple groups.

We illustrate these approaches through explicit realizations in small alternating groups. For the smallest nonabelian simple group, namely the alternating group A_5 , we obtain a commutator-based realization of the NAND gate requiring 8 group operations. This improves the operation count of the recent construction of Nuida [10], which uses compression functions over groups and requires 10 group multiplications to realize a NAND gate. In addition, we obtain a direct commutator-based realization of the AND gate in A_5 requiring 8 group operations, a functionality not explicitly provided in [10]. Its companion XOR gate over $\mathbb{F}_2$ achieves the same operation count as the construction in [10], namely 10 group operations. We also give a different realization of the AND gate that

uses only group multiplications, requiring 9 group multiplications in total; its companion XOR gate is then realized by a single group multiplication.

Our framework also yields particularly efficient realizations in larger simple groups. In the group A_6 (and similarly in S_6), we obtain an AND gate requiring only 5 group multiplications and an XOR gate requiring a single group multiplication. This viewpoint provides a conceptual explanation for the encoding over S_6 described in [4], which is obtained there by modifying an S_7 construction by hand.

Outline of the paper The remainder of this paper is organized as follows. Section 2 introduces the required group-theoretic notation and the notion of a homomorphic encoding over a group. Section 3 presents a commutator-based realization of a NAND gate, including an explicit construction in A_5 . Section 4 gives realizations of the XOR and AND gates over $\mathbb{F}_2$ with parameters in A_5 and A_6 . Section 5 concludes the paper.

2 Background and Notation

Unless specified otherwise, all groups considered in this paper are finite, written multiplicatively, and their identity element is denoted by e .

2.1 Elements of Group Theory

We recall several standard notions from group theory; a general reference is [13].

Basic notions Let G be a group. A subgroup $N \leq G$ is said to be *normal*, written $N \trianglelefteq G$, if it is invariant under conjugation, that is, if $g N g^{-1} = N$ for all $g \in G$. Equivalently, $g n g^{-1} \in N$ for all $g \in G$ and $n \in N$. The *center* of G is the subgroup $Z(G) := \{z \in G \mid z g = g z \text{ for all } g \in G\}$, consisting of all elements that commute with every element of G . For $g, h \in G$, the (group) *commutator* of g and h is

$$[g, h] := g h g^{-1} h^{-1} .$$

Thus $[g, h] = e$ if and only if g and h commute.

Finally, a nontrivial group G is called *simple* if its only normal subgroups are $\{e\}$ and G itself.

Permutation groups For a positive integer n , the *symmetric group* S_n is the group of all permutations of the set $\{1, \dots, n\}$, with group multiplication given by composition. Its elements are bijections $\sigma: \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, and $\#S_n = n!$. Permutations in S_n are written in *cycle notation*. A cycle $(i_1 i_2 \dots i_k)$ denotes the permutation sending $i_1 \mapsto i_2, i_2 \mapsto i_3, \dots, i_k \mapsto i_1$, and fixing all other elements of $\{1, \dots, n\}$. The identity permutation is denoted by $()$. Disjoint cycles commute, and every permutation can be written uniquely (up to the order of disjoint cycles) as a product of disjoint cycles. A 2-cycle is called a *transposition*.

The *alternating group* A_n is the subgroup of S_n consisting of all even permutations, that is, permutations that can be written as a product of an even number of transpositions. It is a normal subgroup of S_n of index 2, and hence $\#A_n = n!/2$. For $n \geq 5$, the group A_n is nonabelian and simple.

2.2 Fully Homomorphic Encodings over Groups

We formalize the notion of realizing arithmetic inside a group.

Definition 1. *Let $(\mathcal{M}, +, \cdot)$ be a commutative ring with 1 and let G be a group. A fully homomorphic encoding over G is an injective map*

$$\text{Ecd}: \mathcal{M} \longrightarrow G$$

together with efficiently computable operations

$$\text{Add}, \text{Mult}: G \times G \rightarrow G$$

such that for all $m_1, m_2 \in \mathcal{M}$,

$$\text{Add}(\text{Ecd}(m_1), \text{Ecd}(m_2)) = \text{Ecd}(m_1 + m_2)$$

and

$$\text{Mult}(\text{Ecd}(m_1), \text{Ecd}(m_2)) = \text{Ecd}(m_1 \cdot m_2) .$$

We illustrate this notion with the realization in $G = A_6$ described in Section 4.4. There the message space is $\mathcal{M} = \mathbb{F}_2$, identified with the set $\{0, 1\}$ equipped with addition $\oplus$ (XOR) and multiplication $\wedge$ (AND). The encoding is given by

$$\text{Ecd}(0) = () , \quad \text{Ecd}(1) = (1\ 2)(3\ 4) .$$

If $x = \text{Ecd}(\beta)$ and $x' = \text{Ecd}(\beta')$ for $\beta, \beta' \in \mathbb{F}_2$, then

$$\text{Add}(x, x') = \text{XOR}(x, x') , \quad \text{Mult}(x, x') = \text{AND}(x, x') ,$$

where XOR is realized by group multiplication and AND by a commutator-based construction.

3 Boolean Computation via a NAND Gate

We denote the Boolean NAND operator by $\bar{\wedge}$, defined by

$$u \bar{\wedge} v = \neg(u \wedge v) = 1 - uv , \quad u, v \in \{0, 1\} .$$

Since the NAND gate is functionally complete, it suffices to realize NAND in order to obtain universal Boolean computation. In this section, we present a construction in which NAND is implemented by a *single commutator* in a finite nonabelian simple group, and we give an explicit instantiation in A_5 .

3.1 Commutator-Based NAND Gate

Let G be a finite nonabelian simple group. Select two distinct nontrivial elements $z_0, z_1 \in G$ that encode the bits

$$0 \longleftrightarrow z_0, \quad 1 \longleftrightarrow z_1.$$

The requirement that both encoding elements be nontrivial is essential: if $z_0 = e$, then any commutator involving z_0 is trivial, so NAND cannot be realized as a single commutator.

Fix elements $g, h \in G$ and define

$$\text{NAND}(x, y) := [g x g^{-1}, h y h^{-1}], \quad x, y \in \{z_0, z_1\}. \quad (1)$$

This map depends only on intrinsic group operations.

Correctness conditions Let

$$a_i = g z_i g^{-1}, \quad b_j = h z_j h^{-1}.$$

The map Eq. (1) realizes the Boolean NAND gate under the encoding $0 \leftrightarrow z_0$ and $1 \leftrightarrow z_1$, provided that

$$[a_i, b_j] = z_i \bar{\wedge} j, \quad i, j \in \{0, 1\}.$$

Thus the construction reduces to finding four elements a_0, a_1, b_0, b_1 in prescribed conjugacy classes whose commutator table matches the Boolean NAND table. Ore's theorem guarantees that every element of G is a commutator; hence z_0 and z_1 both occur in the image of the commutator map. The present construction strengthens this requirement by imposing simultaneous constraints on four specific commutators arising from fixed conjugacy classes.

Universality Once such elements z_0, z_1, g, h are found, the commutator map

$$\text{NAND}(x, y) = [g x g^{-1}, h y h^{-1}]$$

realizes the Boolean NAND operation under the given encoding. Since NAND is functionally complete, arbitrary Boolean functions can then be implemented by composing this operation.

3.2 An Explicit NAND Gate in A_5

We now give a concrete realization that implements a Boolean NAND gate entirely within the group law of $G = A_5$.

Define

$$z_0 = (12345), \quad z_1 = (13542), \quad g = (24)(35), \quad h = (153).$$

The elements z_0 and z_1 are both 5-cycles and therefore lie in the same conjugacy class of A_5 . Moreover, g is an involution ($g^{-1} = g$), while h has order 3 (so $h^{-1} = (135)$).

Let $a_i = g z_i g^{-1}$ and $b_j = h z_j h^{-1}$. A direct computation in A_5 yields the commutator table

$[a_i, b_j]$	$j = 0$	$j = 1$
$i = 0$	z_1	z_1
$i = 1$	z_1	z_0

which coincides with the Boolean NAND table under the encoding $0 \leftrightarrow z_0$ and $1 \leftrightarrow z_1$.

Expanding the commutator operator, the NAND gate can be evaluated using 8 group operations in A_5 as

$$\text{NAND}(x, y) = U \cdot V \cdot (V \cdot U)^{-1} \quad \text{where} \quad \begin{cases} U = (24)(35) \cdot x \cdot (24)(35) \\ V = (153) \cdot y \cdot (135) \end{cases}.$$

4 XOR and AND Gates over $\mathbb{F}_2$

Let G be a finite nonabelian simple group. We now turn from universal Boolean computation to the direct realization of the arithmetic gates over $\mathbb{F}_2$, namely

$$x \oplus y = x + y \pmod{2} \quad \text{and} \quad x \wedge y = x \cdot y \pmod{2}.$$

In contrast to the NAND construction of Section 3, it is here natural to adopt the canonical encoding

$$0 \longleftrightarrow e, \quad 1 \longleftrightarrow z_1.$$

so that each bit is represented by an element of the set $\{e, z_1\} \subseteq G$.

4.1 Commutator-Based AND Gate

Fix $g, h \in G$ and define

$$\text{AND}(x, y) := [g x g^{-1}, h y h^{-1}], \quad x, y \in \{z_0, z_1\}.$$

Let $a_i = g z_i g^{-1}$ and $b_j = h z_j h^{-1}$. Under the encoding $0 \leftrightarrow e$ and $1 \leftrightarrow z_1$, this realizes the Boolean AND gate provided

$$[a_i, b_j] = z_{i \wedge j}, \quad i, j \in \{0, 1\}.$$

Because $z_0 = e$, we automatically have $a_0 = b_0 = e$, and therefore

$$[a_0, b_0] = e, \quad [a_0, b_1] = e, \quad [a_1, b_0] = e,$$

which match the three zero-outputs of the AND table. The only nontrivial constraint is

$$[a_1, b_1] = z_1,$$

that is, two conjugates of z_1 must have commutator equal to z_1 .

Remark 1. Suppose z_1 has order 2. If a_1, b_1 are conjugates of z_1 , they are involutions as well, and hence

$$[a_1, b_1] = a_1 b_1 a_1^{-1} b_1^{-1} = a_1 b_1 a_1 b_1 = (a_1 b_1)^2 .$$

Thus the condition $[a_1, b_1] = z_1$ becomes $(a_1 b_1)^2 = z_1$, so $T := a_1 b_1$ must have order 4. In particular, G must contain two involutions whose product has order 4, and hence a subgroup isomorphic to the dihedral group D_8 .

When the condition of Remark 1 is not satisfied, it may still be possible to realize the AND gate by using a product of two commutators. For instance, one may consider constructions of the form

$$\text{AND}(x, y) := [g_1 x g_1^{-1}, h_1 y h_1^{-1}] \cdot [g_2 x g_2^{-1}, h_2 y h_2^{-1}] ,$$

which can satisfy the required Boolean table even when a single commutator does not suffice. Products of such commutator maps also appear in structural expressiveness results such as [5, Lemma 6.3].

4.2 XOR Gate over $\mathbb{F}_2$

We present two XOR realizations, depending on the order of z_1 .

XOR via group multiplication With the same encoding and z_1 of order 2, group multiplication directly implements addition in $\mathbb{F}_2$:

$$z_0 \cdot z_0 = z_0 , \quad z_0 \cdot z_1 = z_1 \cdot z_0 = z_1 , \quad z_1 \cdot z_1 = z_0 .$$

Hence

$$\text{XOR}(x, y) := x \cdot y$$

realizes the XOR gate.

Commutator-based XOR gate If G contains a subgroup isomorphic to D_8 , then it contains an element $z_1 = [a_1, b_1]$ of order 2, and the above one-multiplication implementation should be preferred.

If G has no involutions, then it contains an element of order 3. Indeed, by the classification of finite simple groups, the only nonabelian finite simple groups whose order is not divisible by 3 are the Suzuki groups $\text{Sz}(2^{2n+1})$, and these contain dihedral subgroups of order 8, hence involutions (so the contrapositive implies that a simple group without involutions must have order divisible by 3).

When $z_0 = e$, a direct commutator-based realization of the form $\text{XOR}(x, y) = [g x g^{-1}, h y h^{-1}]$ is impossible, since XOR requires $[a_0, b_1] = [a_1, b_0] = z_1$, but $a_0 = e$ implies $[a_0, b_1] = e \neq z_1$. If z_1 has order 3, however, XOR can be obtained from the commutator-based AND gate via

$$\text{XOR}(x, y) := x \cdot y \cdot \text{AND}(x, y) ,$$

since $z_1^3 = e$ and a direct verification shows that the Boolean XOR table is satisfied.

4.3 Fully Homomorphic Encoding of $\mathbb{F}_2$

Assume that $z_0 = e$ and $z_1 \in G$ are chosen so that the above construction realizes the AND gate, and that XOR is implemented either by group multiplication (when z_1 has order 2) or via $\text{XOR}(x, y) = x \cdot y \cdot \text{AND}(x, y)$ when z_1 has order 3.

Then the map

$$\text{Ecd}(0) = e, \quad \text{Ecd}(1) = z_1$$

together with $\text{Add} = \text{XOR}$ and $\text{Mult} = \text{AND}$ forms a fully homomorphic encoding of $\mathbb{F}_2$ over G .

4.4 Explicit Realizations

We now give explicit realizations in small simple groups.

A first explicit realization in A_5 In A_5 , involutions are double transpositions. The product of two double transpositions has order 1, 2, 3, or 5, never 4. Hence $(a_1 b_1)^2$ cannot be an involution, and therefore no element $z_1 = [a_1, b_1]$ of order 2 arises in this manner (see Remark 1).

The XOR realization via group multiplication therefore does not apply. We give below concrete parameters for a commutator-based implementation in group $G = A_5$.

Define

$$z_0 = () , \quad z_1 = (1\,2\,3) , \quad g = (1\,5\,4) , \quad h = (1\,2)(3\,4) .$$

Here z_1 is a 3-cycle and thus has order 3. A direct computation yields $a_1 = g z_1 g^{-1} = (2\,3\,5)$ and $b_1 = h z_1 h^{-1} = (1\,4\,2)$, and in turn

$$[a_1, b_1] = a_1 b_1 a_1^{-1} b_1^{-1} = (1\,2\,3) = z_1 .$$

Because $z_0 = ()$ (that is, $z_0 = e$), the remaining three entries of the AND table are automatically satisfied. Hence

$$\text{AND}(x, y) = [g x g^{-1}, h y h^{-1}]$$

realizes the Boolean AND gate in A_5 under the encoding $0 \leftrightarrow ()$, $1 \leftrightarrow (1\,2\,3)$. Since z_1 has order 3, XOR is obtained via

$$\text{XOR}(x, y) = x \cdot y \cdot \text{AND}(x, y) .$$

Regarding the operation count, as in the NAND realization, the AND gate requires 8 group operations (7 group multiplications and 1 group inversion), and the XOR gate requires 2 additional group multiplications.

Another realization of the AND gate requiring 9 group multiplications in the multiplication-only model (i.e., disallowing explicit inversion) is described below, together with a matching XOR gate implemented by a single group multiplication.

Remark 2. The same obstruction holds in the symmetric group S_5 . Its involutions are either transpositions or double transpositions, and in both cases the product of two conjugate involutions never has order 4. Consequently, S_5 likewise admits no realization with $z_1 = [a_1, b_1]$ of order 2, and one must again rely on the commutator-based constructions for AND and XOR.

A second explicit realization in A_5 We now present an explicit realization of the above type in the group $G = A_5$, using a product of two commutators. Define

$$\begin{aligned} z_0 = () , \quad z_1 = (12)(34) , \quad g_1 = (15)(23) , \quad h_1 = (12)(45) , \\ g_2 = (25)(34) , \quad h_2 = g_2 g_1^{-1} h_1 = (14)(35) . \end{aligned}$$

The choice of h_2 is made so that $g_1^{-1} h_1 = g_2^{-1} h_2 = (13254)$, which allows a common factor to be extracted in the evaluation of the two commutators. Writing

$$T = x \cdot (13254) \cdot y ,$$

the AND gate can then be evaluated as

$$\text{AND}(x, y) = ((15)(23) \cdot T \cdot (12)(45))^2 \cdot ((25)(34) \cdot T \cdot (14)(35))^2 .$$

A direct verification shows that this map realizes the Boolean AND gate under the encoding $0 \leftrightarrow ()$ and $1 \leftrightarrow (12)(34)$. Since z_1 has order 2, the companion XOR gate is simply given by

$$\text{XOR}(x, y) = x \cdot y .$$

This realization uses only group multiplications. More precisely, the AND gate requires 9 group multiplications, while XOR is implemented by a single group multiplication.

An explicit realization in A_6 The situation changes in the group $G = A_6$, which contains a subgroup isomorphic to D_8 (and therefore so does A_n for all $n \geq 6$, since A_6 embeds in A_n as the subgroup fixing $\{7, \dots, n\}$ pointwise).

Define

$$z_0 = () , \quad z_1 = (12)(34) , \quad g = (23)(56) , \quad h = (15)(26) .$$

All elements lie in A_6 , and z_1 is an involution. A direct computation yields $g z_1 g^{-1} = (13)(24)$ and $h z_1 h^{-1} = (34)(56)$. These are involutions whose product, $(13)(24) \cdot (34)(56) = (1324)(56)$, is a cycle of type $(4, 2)$, and hence has order 4. Consequently,

$$\langle (13)(24), (34)(56) \rangle \cong D_8 .$$

Moreover, one has $[g z_1 g^{-1}, h z_1 h^{-1}] = [(13)(24), (34)(56)] = (12)(34) = z_1$. Thus the requirement for realizing the Boolean AND gate in A_6 under the encoding $0 \leftrightarrow ()$, $1 \leftrightarrow (12)(34)$ is satisfied.

Since z_1 has order 2, the XOR gate is obtained directly via group multiplication: $\text{XOR}(x, y) = x \cdot y$.

Since z_0 and z_1 are both involutions, so are their conjugates. In turn, the commutator-based AND becomes $\text{AND}(x, y) = g x g^{-1} h y h^{-1} g x g^{-1} h y h^{-1}$, for $x, y \in \{z_0, z_1\}$. Noting that $g^{-1} h = (16325)$, the AND gate can be evaluated using 5 group multiplications as

$$\text{AND}(x, y) = ((23)(56) \cdot x \cdot (16325) \cdot y \cdot (15)(26))^2.$$

The XOR gate $\text{XOR}(x, y) = x \cdot y$ requires only a single group multiplication.

5 Conclusion

This paper revisits the problem of realizing arithmetic over $\mathbb{F}_2$ within finite nonabelian simple groups using only intrinsic group operations. Two complementary approaches are presented: one based on realizing a universal Boolean gate (NAND), and one based on direct realizations of the arithmetic operations XOR and AND.

Explicit realizations in the alternating groups A_5 and A_6 illustrate different aspects of these approaches. In A_5 , compact commutator-based implementations of Boolean gates are obtained. In A_6 , the arithmetic operations admit particularly simple realizations, requiring only a very small number of group multiplications. This latter example also sheds light on the encoding over S_6 that appears in recent work.

References

1. Barrington, D.A.M.: Bounded-width polynomial-size branching programs recognize exactly those languages in NC^1 . In: 18th ACM Symposium on Theory of Computing (STOC '86). pp. 1–5 (1986). <https://doi.org/10.1145/12130.12131>
2. Ben-Or, M., Cleve, R.: Computing algebraic formulas using a constant number of registers. *SIAM Journal on Computing* **21**(1), 54–58 (1992). <https://doi.org/10.1137/0221006>
3. Grigoriev, D., Ponomarenko, I.: On non-abelian homomorphic public-key cryptosystems. *Journal of Mathematical Sciences* **126**(3), 1158–1166 (2005). <https://doi.org/10.1007/s10958-005-0077-3>
4. Guillot, P., Hoang Duc, A., Koskas, M., Méhats, F.: Introducing GRAFHEN: GRoup-bAsed fully homomorphic encryption without noise. *Cryptology ePrint Archive, Paper 2025/1907* (2025), <https://ia.cr/2025/1907>
5. Khamsemanan, N., Ostrovsky, R., Skeith, W.E.: On the black-box use of somewhat homomorphic encryption in noninteractive two-party protocol. *SIAM Journal on Discrete Mathematics* **30**(1), 266–295 (2016). <https://doi.org/10.1137/110858835>, an earlier version appears in [?]

6. Krohn, K., Maurer, W.D., Rhodes, J.: Realizing complex Boolean functions with simple groups. *Information and Control* **9**, 190–195 (1966). [https://doi.org/10.1016/S0019-9958\(66\)90229-4](https://doi.org/10.1016/S0019-9958(66)90229-4)
7. Li, J., Wang, L.: Noiseless fully homomorphic encryption. *Cryptology ePrint Archive*, Paper 2017/839 (2017), <https://ia.cr/2017/839>
8. Liebeck, M.W., O’Brien, E.A., Shalev, A., Tiep, P.H.: The Ore conjecture. *Journal of the European Mathematical Society* **12**(4), 939–1008 (2010). <https://doi.org/10.4171/JEMS/220>
9. Maurer, W.D., Rhodes, J.L.: A property of finite simple non-abelian groups. *Proceedings of the American Mathematical Society* **16**, 552–554 (1965). <https://doi.org/10.1090/S0002-9939-1965-0175971-0>
10. Nuida, K.: On compression functions over groups with applications to homomorphic encryption. *Journal of Algebra and Its Applications*. <https://doi.org/10.1142/S0219498827500642>, to appear
11. Nuida, K.: Towards constructing fully homomorphic encryption without ciphertext noise from group theory. In: Takagi, T., et al. (eds.) *International Symposium on Mathematics, Quantum Theory, and Cryptography (MQC 2019)*. *Mathematics for Industry*, vol. 33, pp. 57–78. Springer (2021). https://doi.org/10.1007/978-981-15-5191-8_8
12. Rappe, D.K.: Homomorphic Cryptosystems and their Applications. Ph.D. thesis, Universität Dortmund (2004). <https://doi.org/10.17877/DE290R-15728>
13. Robinson, D.J.S.: *A Course in the Theory of Groups*, Graduate Texts in Mathematics, vol. 80. Springer, 2nd edn. (1996). <https://doi.org/10.1007/978-1-4419-8594-1>